

Cloud Computing Tools

Taller IV: Pipeline de visualización sobre EC2 con Docker, Grafana y Amazon RDS

Análisis geoespacial de gasolineras con Foursquare Places API

Juan Pablo Jiménez — Maestría en Analítica Aplicada
Harrison Alonso — Maestría en Analítica Aplicada
Juan Fernando Serrano — Maestría en Analítica Aplicada

Universidad de La Sabana
Prof.: Hugo Franco, Ph.D.

9 de septiembre de 2026

1 Contexto

La cadena de restaurantes que hemos venido analizando (Chía, Cundinamarca) busca priorizar zonas para su próxima expansión. En los talleres anteriores exploramos la densidad de restaurantes por categoría (Foursquare, categoría *restaurants*) y la relación entre condiciones meteorológicas y afluencia. Un tercer factor, aún no evaluado, es la cercanía a nodos de alto tráfico vehicular: las gasolineras concentran paso constante de conductores y suelen actuar como polos de consumo asociado (comida rápida, cafeterías, tiendas de conveniencia) en su radio inmediato.

Este taller materializa ese análisis sobre una arquitectura de nube desplegada de forma programática. Foursquare Places expone la categoría *Gas Station / Garage* (id `4bf58dd8d48988d113951735`) con las mismas coordenadas y atributos que ya usamos para restaurantes, lo que permitió construir la ingesta con el mismo patrón ya validado, cambiando únicamente el parámetro de categoría.

La restricción operativa central es que el ejercicio corre sobre AWS Academy Learner Lab, con crédito limitado e irre recuperable. Por ello toda instancia EC2 y RDS se enciende únicamente durante la sesión de trabajo y se detiene al terminar.

2 Objetivo

Construir un pipeline de visualización interactiva que, a partir de datos georreferenciados de Foursquare, permita identificar zonas de alto tráfico vehicular desatendidas en oferta gastronómica en el casco urbano de Chía, desplegando la capa de visualización (Grafana) sobre una instancia EC2 aprovisionada programáticamente con `boto3` e Infraestructura como Código, y conectada a Amazon RDS como fuente de datos nativa.

Pregunta analítica: ¿qué gasolineras del casco urbano de Chía tienen, dentro de un radio de 500 m, un número bajo de restaurantes registrados en Foursquare, y por lo tanto representan zonas de alto tráfico con baja oferta gastronómica competitiva?

3 Arquitectura implementada

El pipeline consta de cuatro capas: ingesta (Python + Foursquare Places API), almacenamiento (Amazon RDS PostgreSQL), cómputo de visualización (EC2 + Docker + Grafana) y exposición de red (Security Groups + mapeo de puertos).

Tabla 1: Rol de cada componente y su control de costo asociado.

Servicio	Rol en el pipeline	Control de costos (FinOps)
Foursquare Places API	Fuente de datos: búsqueda por categoría <i>Gas Station / Garage</i> alrededor de (4.85876, -74.05866), radio 5 000 m.	Sin costo AWS asociado; se limita el parámetro <code>limit</code> y el radio para no agotar la cuota de la API.
Script local (boto3)	Aprovisiona la instancia EC2 e inyecta el bloque <code>UserData</code> con la instalación autónoma de Docker y el despliegue del contenedor.	Ejecución local, sin costo; la instancia creada es <code>t2.micro</code> (capa gratuita).
Amazon RDS (PostgreSQL)	Almacena las tablas <code>gasolineras</code> y <code>datos_externos</code> ; ejecuta el join espacial mediante fórmula de Haversine en SQL.	Instancia <code>db.t3.micro</code> encendida solo durante la sesión de análisis y detenida (<i>Stop</i>) al terminar.
Amazon EC2 + Docker	Host del contenedor de Grafana, instalado automáticamente en el arranque vía <code>UserData</code> .	Se detiene inmediatamente al terminar la sesión de trabajo.
Grafana (contenedor)	Capa de visualización; consume RDS como <i>data source</i> nativo y renderiza los paneles del dashboard.	Contenedor efímero; la imagen se descarga una sola vez al arranque.

4 Implementación

4.1 Aprovisionamiento programático de EC2 con boto3

Se desarrolló localmente el script `docker_deployment.py`, que instancia el servidor virtual e inyecta el bloque de Infraestructura como Código. El AMI se resuelve dinámicamente vía SSM en lugar de codificarlo, lo que evita que el script se rompa cuando AWS publica una nueva versión de Amazon Linux 2023:

Código 1: Resolución dinámica del AMI y creación de la instancia.

```
ssm = session.client('ssm')
AMI_ID = ssm.get_parameter(
    Name='/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64'
)['Parameter']['Value']

instances = ec2.create_instances(
    ImageId=AMI_ID,
    MinCount=1, MaxCount=1,
    InstanceType="t2.micro",
    KeyName=KEYPAIR_NAME,
    UserData=user_data_script,
    SecurityGroupIds=SG_IDS,
)
```

4.2 Bloque UserData: instalación autónoma de Docker

El bloque `UserData` se ejecuta una única vez en el primer arranque del sistema, mediante `cloud-init`. Actualiza los repositorios, instala el Docker Engine con el gestor de paquetes nativo (`dnf` en Amazon Linux 2023), habilita el servicio en el arranque y gestiona los grupos de permisos:

Código 2: Bloque de Infraestructura como Código inyectado en el arranque.

```
#!/bin/bash
# 1. Actualizar el sistema e instalar Docker
dnf update -y
dnf install docker -y

# 2. Iniciar el servicio y habilitarlo en el arranque
systemctl start docker
systemctl enable docker

# 3. Permitir ejecutar docker sin sudo constante
usermod -aG docker ec2-user

# 4. Desplegar Grafana con aislamiento de puertos
docker run -d \
  --name=grafana_server \
  --restart unless-stopped \
  -p 3000:3000 \
  grafana/grafana:latest
```

La instrucción `usermod -aG docker ec2-user` agrega el usuario al grupo `docker`, lo que permite ejecutar comandos del motor sin privilegios de superusuario en cada invocación. La bandera `-restart unless-stopped` garantiza que el contenedor se vuelva a levantar automáticamente si la instancia se reinicia.

4.3 Aislamiento de puertos y reglas de red

El mapeo `-p 3000:3000` vincula el puerto interno del contenedor (aislado en su propio espacio de red) con el puerto del sistema operativo anfitrión. Ese mapeo por sí solo no basta: el tráfico externo sigue bloqueado hasta que el Security Group lo permita explícitamente. Se configuraron dos reglas de entrada:

Tabla 2: Reglas de entrada configuradas en el Security Group.

Tipo	Protocolo	Puerto	Origen
SSH	TCP	22	My IP
Custom TCP	TCP	3000	0.0.0.0/0
PostgreSQL ¹	TCP	5432	IP del host de análisis /32

¹Esta regla se configura en el Security Group de la instancia RDS, no en el de EC2.

4.4 Conexión remota y verificación

El acceso a la instancia se realizó por SSH con el par de claves generado en AWS. Un punto relevante de la operación: el par de claves se asocia a la instancia únicamente en el momento del lanzamiento (parámetro `KeyName`), cuando AWS inyecta la clave pública en `~/.ssh/authorized_keys`. Un mismo par puede reutilizarse para múltiples instancias, pero una instancia solo acepta el par con el que fue lanzada.

Código 3: Restricción de permisos de la llave y conexión (PowerShell).

```
icacls .\llave-cloud-docker.pem /reset
icacls .\llave-cloud-docker.pem /inheritance:r
icacls .\llave-cloud-docker.pem /grant:r "$($env:USERNAME):(R)"

ssh -i .\llave-cloud-docker.pem ec2-user@<PUBLIC_IP>
```

4.5 Ingesta de datos y carga a RDS

Se implementó el script `gasolineras_pipeline.py`, que consulta la API de Foursquare, crea la tabla si no existe y carga los datos de forma idempotente. Cabe señalar que Foursquare migró su API: el endpoint actual es `places-api.foursquare.com` y requiere autenticación `Bearer` más un encabezado de versión.

Código 4: Consulta a la API de Foursquare por categoría.

```
headers = {
    "Authorization": f"Bearer {FOURSQUARE_API_KEY}",
    "X-Places-API-Version": "2025-06-17",
}
params = {
    "ll": "4.85876,-74.05866",
    "radius": 5000,
    "fsq_category_ids": "4bf58dd8d48988d113951735", # Gas Station / Garage
    "limit": 50,
}
```

La carga a RDS usa `ON CONFLICT ... DO UPDATE`, lo que permite reejecutar el script sin duplicar registros (idempotencia).

4.6 Registro de RDS como data source en Grafana

Accediendo a `http://<PUBLIC_IP>:3000` y autenticando en Grafana, se registró Amazon RDS como fuente de datos nativa con los parámetros: `Host` igual al endpoint de RDS con sufijo `:5432`, base de datos `postgres`, y modo TLS/SSL en `require`.

5 Consultas analíticas y resultados

5.1 Consulta maestra: cruce espacial

El indicador central se calcula con un producto cartesiano entre ambas tablas, filtrando por distancia de Haversine. Se optó por esta fórmula en lugar de PostGIS para evitar la dependencia de una extensión adicional en la instancia RDS:

Código 5: Ranking de oportunidad por gasolinera.

```
WITH cruce AS (
  SELECT
    g.name AS gasolinera, g.latitude, g.longitude,
    6371000 * acos(LEAST(1.0,
      cos(radians(g.latitude)) * cos(radians(r.latitude)) *
      cos(radians(r.longitude) - radians(g.longitude)) +
      sin(radians(g.latitude)) * sin(radians(r.latitude)))) AS metros
  FROM gasolineras g
  CROSS JOIN datos_externos r
)
SELECT
  gasolinera, latitude, longitude,
  COUNT(*) FILTER (WHERE metros <= 500) AS competidores_500m,
  ROUND(MIN(metros)::numeric, 0) AS dist_competidor_mas_cercano_m,
  CASE
    WHEN COUNT(*) FILTER (WHERE metros <= 500) = 0 THEN 'Alta oportunidad'
    WHEN COUNT(*) FILTER (WHERE metros <= 500) <= 2 THEN 'Oportunidad media'
    ELSE 'Zona saturada'
  END AS clasificacion
FROM cruce
GROUP BY gasolinera, latitude, longitude
ORDER BY competidores_500m ASC, dist_competidor_mas_cercano_m DESC;
```

5.2 Dashboard resultante

El dashboard *Análisis gasolineras-restaurantes* se compone de tres paneles *Stat* con los indicadores agregados, un panel *Geomap* que superpone la capa de gasolineras sobre la cartografía de Chía, y un panel *Table* con el ranking completo. La Figura 1 muestra la vista ordenada de forma ascendente por número de competidores, es decir, encabezada por las ubicaciones con menor competencia registrada.

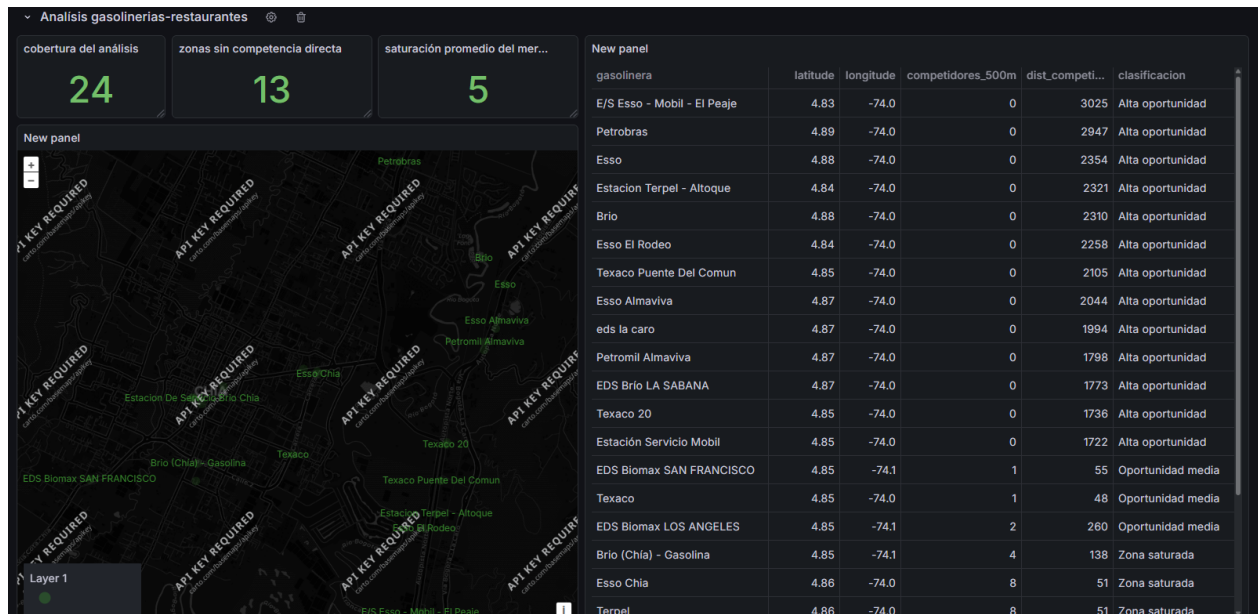


Figura 1: Dashboard interactivo servido desde el contenedor de Grafana en EC2 (http://<PUBLIC_IP>:3000), con Amazon RDS registrado como *data source* nativo. Vista ordenada ascendentemente por competidores en 500 m.

La Figura 2 presenta el mismo dashboard con el orden invertido, lo que expone el extremo opuesto del gradiente: las estaciones del casco urbano con mayor densidad competidora. El contraste entre ambas vistas es el que permite leer el patrón radial descrito en la Sección 6.1.

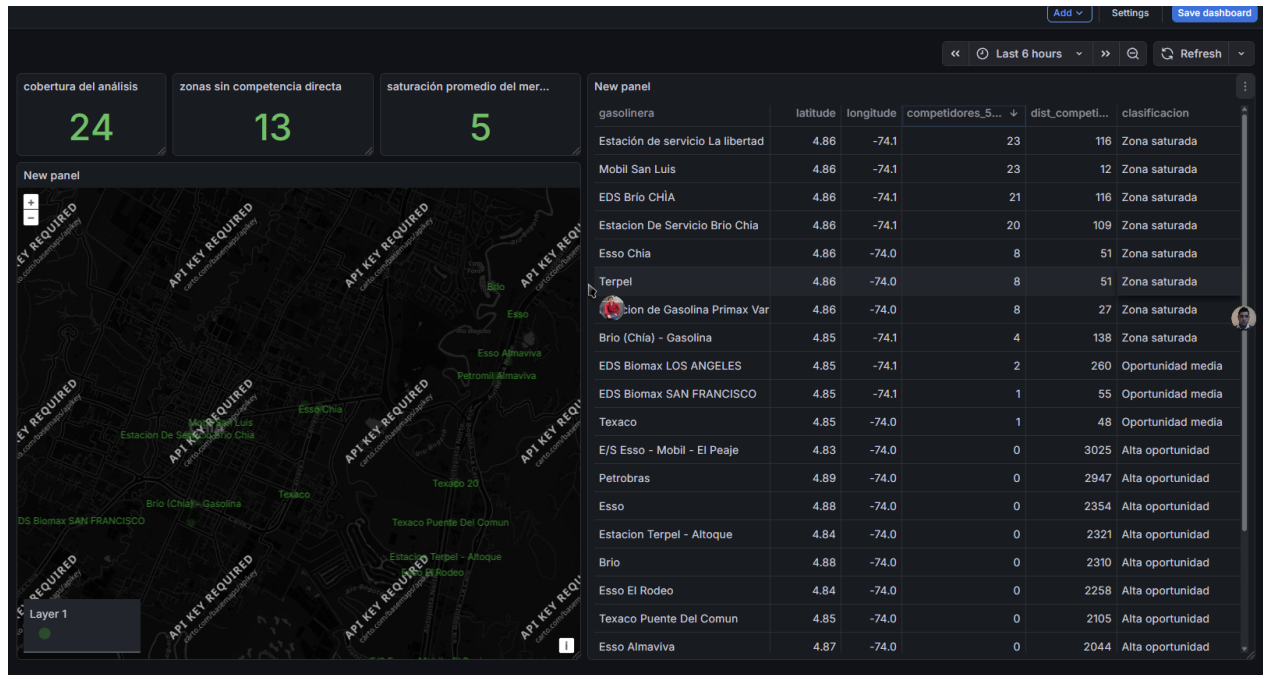


Figura 2: Vista del mismo dashboard ordenada descendientemente, encabezada por las estaciones en zona saturada del casco urbano de Chía.

5.3 Indicadores clave del dashboard

Tabla 3: KPIs expuestos en el dashboard.

Indicador	Valor	Lectura de negocio
Cobertura del análisis (gasolineras)	24	Nodos de tráfico vehicular evaluados.
Zonas sin competencia directa	13	Gasolineras sin restaurantes en 500 m.
Saturación promedio del mercado	5	Restaurantes promedio por buffer de 500 m.

5.4 Ranking de oportunidad

Tabla 4: Extracto del ranking, ordenado de mayor a menor saturación.

Gasolinera	Comp. 500 m	Dist. mín. (m)	Clasificación
Estación de servicio La Libertad	23	116	Zona saturada
Mobil San Luis	23	12	Zona saturada
EDS Brío Chía	21	116	Zona saturada
Estación De Servicio Brío Chía	20	109	Zona saturada
Esso Chía	8	51	Zona saturada
Estación Primax Variante Chía	8	27	Zona saturada
Brío (Chía)	4	138	Zona saturada
EDS Biomax Los Ángeles	2	260	Oportunidad media
EDS Biomax San Francisco	1	55	Oportunidad media
Texaco	1	48	Oportunidad media
Terpel	1	28	Oportunidad media
E/S Esso - Mobil El Peaje	0	3 025	Alta oportunidad ²
Petrobras	0	2 947	Alta oportunidad*
Texaco Puente del Común	0	2 105	Alta oportunidad*

6 Análisis e interpretación

6.1 Hallazgo principal: gradiente de saturación radial

Los datos revelan un patrón claro: la saturación competitiva decrece monótonamente con la distancia al centro de Chía. Las cuatro gasolineras más céntricas (a 308–465 m del centro) concentran entre 20 y 23 restaurantes en su radio de 500 m, mientras que las ubicadas a más de 3 km no registran ninguno. Esto confirma la estructura urbana esperada: la oferta gastronómica se concentra en el casco urbano y se desvanece hacia la periferia.

6.2 Hallazgo crítico: sesgo de cobertura del muestreo

Un análisis de los datos revela una limitación metodológica que condiciona la interpretación del ranking. El conjunto de restaurantes ingerido en talleres anteriores contiene 50 registros cuya distancia máxima al punto de referencia es de **2 182 m**. Las gasolineras, en cambio, se extienden hasta **4 521 m** del mismo punto.

La consecuencia es determinante: **las 14 gasolineras clasificadas como “alta oportunidad” se encuentran, sin excepción, a más de 2 182 m del centro**, es decir, fuera del alcance del muestreo de restaurantes. Su conteo de cero competidores no evidencia ausencia real de oferta gastronómica, sino ausencia de datos en esa franja. La distancia al restaurante más cercano en esos casos (entre 1 332 y 3 025 m) confirma que el vecino más próximo pertenece al casco urbano, no a la periferia.

Por lo tanto, el ranking solo admite interpretación válida en la franja donde ambos conjuntos se solapan, aproximadamente hasta 1 700 m del centro. Dentro de esa zona, las candidatas reales son:

²Clasificación sujeta al sesgo de cobertura discutido en la Sección 6.2.

Tabla 5: Candidatas dentro de la zona de cobertura válida.

Gasolinera	Dist. centro (m)	Comp. 500 m	Dist. mín. (m)
Texaco	1 229	1	48
Terpel	1 133	1	28
EDS Biomax Los Ángeles	1 125	2	260

Estas tres constituyen el *top* de expansión defendible: están en zona con cobertura de datos confiable, presentan baja competencia directa y se ubican en el anillo intermedio, donde el tráfico vehicular sigue siendo relevante pero la densidad gastronómica ya ha caído.

6.3 Hallazgo secundario: colisión de nombres en la clave primaria

La API devolvió 26 registros, pero la tabla almacenó 24. La diferencia se explica por la elección de `name` como clave de conflicto en la sentencia `UPSERT`: dos pares de estaciones comparten nombre de marca (*Terpel* y *Petrobras*) en ubicaciones distintas, y la segunda sobrescribió a la primera. Para una siguiente iteración corresponde usar `fsq_place_id` como clave primaria, que es único por establecimiento.

6.4 Métricas clave para la organización

Para la cadena de restaurantes que usará este dashboard, los indicadores accionables son:

- **Competidores en 500 m** — métrica primaria de decisión; valores entre 1 y 3 señalan zonas con tráfico pero sin saturación.
- **Distancia al competidor más cercano** — desempata entre candidatas con igual conteo; mayor distancia implica menor canibalización.
- **Saturación promedio (5 restaurantes por buffer)** — línea base contra la cual contrastar cualquier ubicación candidata.

7 Alcance y limitaciones

El análisis se restringe al casco urbano de Chía y a un único radio de referencia (500 m), elegido por ser una distancia caminable típica desde un punto de reabastecimiento vehicular; sensibilizar ese radio (250 m y 1 km) queda como extensión futura.

La proximidad geográfica es una variable proxy de tráfico y no mide directamente el volumen real de vehículos por gasolinera, dato que Foursquare no expone. El resultado debe leerse como una priorización exploratoria de zonas, no como un estimado de demanda.

La limitación más relevante es el sesgo de cobertura documentado en la Sección 6.2: ambos conjuntos se ingirieron con radios de búsqueda distintos, lo que invalida la comparación en la periferia. Corregirlo requiere reejecutar la ingesta de restaurantes con el mismo radio de 5 000 m usado para gasolinerías.

Finalmente, la API limita cada consulta a 50 resultados; para coberturas mayores sería necesario paginar o segmentar la búsqueda en múltiples centros geográficos.

8 Conclusiones

Se implementó de extremo a extremo un pipeline de visualización sobre AWS, cumpliendo los cinco puntos del enunciado: aprovisionamiento programático de EC2 con `boto3`, inyección de Infraestructura como Código vía `UserData` para instalar Docker autónomamente en el arranque, gestión de grupos de permisos, aislamiento y mapeo de puertos (`-p 3000:3000`) complementado con reglas de Security Group, y conexión de Grafana a Amazon RDS como fuente de datos nativa.

El dashboard resultante materializa la pregunta analítica del proyecto final mediante un cruce espacial entre 24 gasolineras y 50 restaurantes, resuelto íntegramente en SQL con la fórmula de Haversine. El análisis identificó un gradiente de saturación radial consistente con la estructura urbana de Chía, y priorizó tres estaciones (Texaco, Terpel y EDS Biomax Los Ángeles) como candidatas defendibles de expansión.

El aporte metodológico más valioso del ejercicio fue detectar que la clasificación automática de “alta oportunidad” resultaba engañosa por un desajuste en los radios de ingesta entre ambos conjuntos. El hallazgo refuerza un principio operativo: en análisis geoespacial comparativo, los parámetros de muestreo de todas las fuentes deben homologarse antes de cruzarlas, o el resultado confunde ausencia de datos con ausencia de fenómeno.